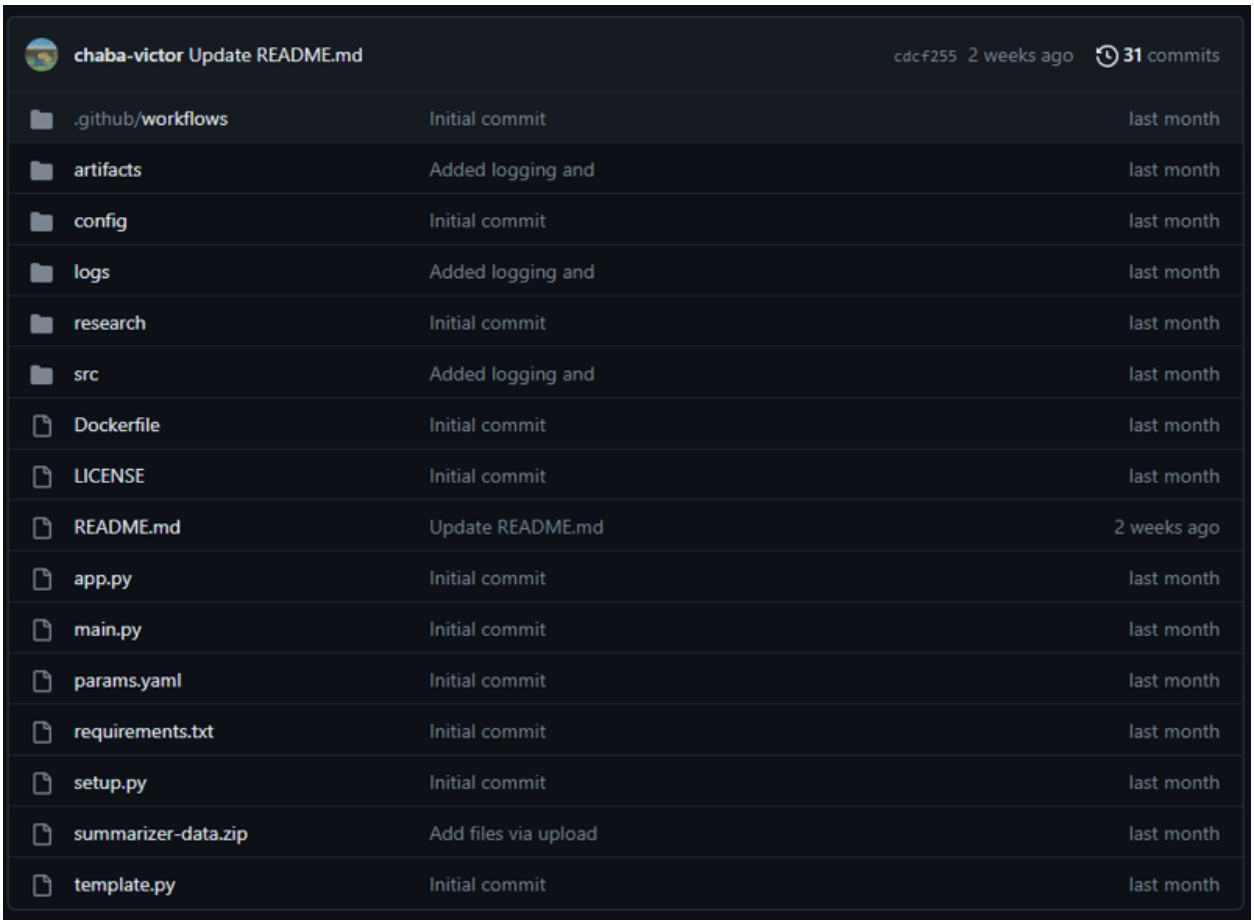
ML Template Project

1. **Project Root**: This is the main folder that contains your entire machine learning project
2. **Data**: This folder is dedicated to storing your datasets and any relevant data files. It can be further divided into subfolders such as:
    a. Raw: Contains the original, unprocessed data files.
    b. Processed: Contains preprocessed data that has undergone cleaning, transformation, and feature engineering.
    c. External: Store any external data sources that you use for your project
3. **Notebooks**: This folder is for Jupyter notebooks or any other interactive notebooks you use for experimentation, analysis, and model development. You can organize it with subfolders like:
    a. Exploratory: Notebook(s) for data exploration and visualization.
    b. Modeling: Notebook(s) for model development, training, and evaluation.
    c. Inference: Notebook(s) for deploying and using trained models for predictions.
4. **Scripts:** This folder contains reusable code scripts or modules that you use in your project. It may include:
    a. Preprocessing: Scripts for data cleaning, transformation, and feature engineering.
    b. Model: Scripts for defining and training machine learning models.
    c. Evaluation: Scripts for model evaluation, metrics calculation, and validation.
    d. Utilities: General-purpose utility scripts or helper functions.
    e. Models: This folder is dedicated to storing trained models or model checkpoints. It can be further organized into subfolders based on different experiments, versions, or architectures.
5. **Documentation**: Include any project-related documentation, such as README files, data dictionaries, or project specifications
6. **Results**: Store output files, reports, or visualizations generated by your models or experiments
7. **Config**: Store configuration files or parameters used in your project, such as hyperparameters, model configurations, or experiment settings
8. **Environment**: Include files related to the project environment, such as requirements.txt or environment.yml, specifying the dependencies and packages required to run your project
9. **Tests**: If you have unit tests or integration tests for your code, you can create a folder to store them
10. Logs: Store log files or output logs generated during training or inference
11. Saved Objects: If your project involves saving intermediate objects or serialized data, such as pickled files or serialized models, you can create a folder to store them

    Template.py

    The template.py file serves as a foundational blueprint or starting point for developing Python code within a machine learning project. It typically contains a set of predefined structures, functions, and placeholders that you can customize and expand upon to build specific functionality. Below is an example that I commonly use. Copy the code, save it as template.py, then run it



The cookiecutter (https://www.cookiecutter.io/templates)
Step 1: pip install cookiecutter
Step 2: cookiecutter -c v1 https://github.com/drivendata/cookiecutter-data-science

https://github.com/kylebradbury/ml-project-structure-demo

Hướng dẫn tạo cấu trúc dữ án mẫu dùng cookiecutter: https://cookiecutter-data-science.drivendata.org/

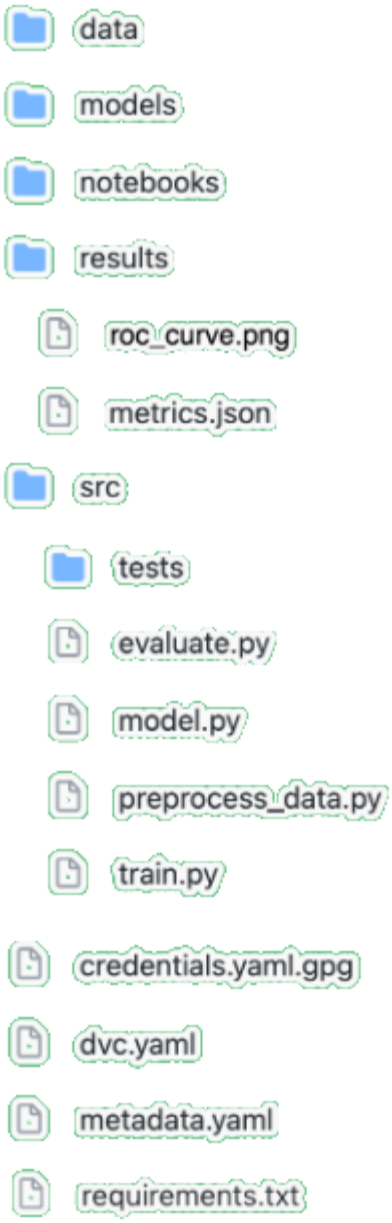
```
├── LICENSE
├── Makefile           <- Makefile with commands like `make data` or `make train`
├── README.md          <- The top-level README for developers using this project.
├── data
│   ├── external       <- Data from third party sources.
│   ├── interim        <- Intermediate data that has been transformed.
│   ├── processed      <- The final, canonical data sets for modeling.
│   └── raw            <- The original, immutable data dump.
│
├── docs               <- A default Sphinx project; see sphinx-doc.org for details
│
├── models             <- Trained and serialized models, model predictions, or model summaries
│
├── notebooks          <- Jupyter notebooks. Naming convention is a number (for ordering),
│                         the creator's initials, and a short `-` delimited description, e.g.
│                         `1.0-jqp-initial-data-exploration`.
│
├── references         <- Data dictionaries, manuals, and all other explanatory materials.
│
├── reports            <- Generated analysis as HTML, PDF, LaTeX, etc.
│   └── figures        <- Generated graphics and figures to be used in reporting
│
├── requirements.txt   <- The requirements file for reproducing the analysis environment, e.g.
│                         generated with `pip freeze > requirements.txt`
│
├── setup.py           <- Make this project pip installable with `pip install -e`
├── src                <- Source code for use in this project.
│   ├── __init__.py    <- Makes src a Python module
│   │
│   ├── data           <- Scripts to download or generate data
│   │   └── make_dataset.py
│   │
│   ├── features       <- Scripts to turn raw data into features for modeling
│   │   └── build_features.py
│   │
│   ├── models         <- Scripts to train models and then use trained models to make
│   │   │                 predictions
│   │   ├── predict_model.py
│   │   └── train_model.py
│   │
│   └── visualization  <- Scripts to create exploratory and results oriented visualizations
│       └── visualize.py
│
└── tox.ini            <- tox file with settings for running tox; see tox.readthedocs.io
```

Project Tools

The main tools discussed in the guide are shown in the following table. As the guide is intended to be modular, a team can swap tools for others depending on the project necessities or preferences.

Tools	Function	Developer
IBM Watson ML	Deploying model as API	IBM
IBM Watson OpenScale	Monitoring Model in production	IBM
DVC	Data and Model Versioning	Iterative
CML	Pipeline Automation	Iterative
Terraform	Setups IBM infrastructure with script	HashiCorp
Github	Code versioning	Github
Github Actions	CI/CD Automation	Github
Pytest	Python script testing	Pytest-dev
Pre-commit	Running tests on local commit	Pre-commit
Cookiecutter	Creating folder structure and files	Cookiecutter

Folder Structure



The above image is the project's folder structure, we'll talk about each specific part in further details trough out the guide

- data , models and results contain files which are being stored and versioned by DVC.
- notebooks contain Jupyter Notebooks used for the exploratory analysis, development of models, or data manipulation.
- src contains scripts for training and evaluating the model as well as tests and scripts for pipelines and APIs

This folder structure is going to be implemented in a blank project in Introduction/Starting a New Project with Cookiecutter (https://mlops-guide.github.io/Structure/starting/)

Requirements

The requirements file is a list of all of a project's dependencies and the specific version of each dependency, including the dependencies needed by the dependencies. It can also be used to create a virtual environment. This is extremely important to avoid conflicts between Python libraries and also ensure the experiments can be reproduced in different machines.

Metadata File

To keep track of the model information we have a metadata.yaml file, this helps with CI/CD and pipeline automation. Such as updating or deploying the model without the need of user input.

Using Jupyter Notebooks vs. Python Scripts

The usage of notebooks may cause some problems for our development cycle:

- Versioning : Since notebooks source code are much more complex , we can't easily visualize the difference between versions using git. There are some tools that can help with that, however
- Reproducibility : A great feature of notebooks is being able to run cells in a non-sequential order, but this is a big problem if we want to reproduce the code, since it's hard to know in what order or which cells where executed, this is especially bad if we want to automate pipeline
- Standardized In/Out : By using scripts we can create pipelines with standardized entries and exits, therefore, we can create universal pipelines since no matter the model what it will receive and return will be in the same format
- Access to Functions : In the model.py script, we define the train and evaluate function, where the model is declared and trained and the metrics for the evaluation are defined. These functions can be called by other scripts such as train.py and evaluate.py so we can create pipelines to train the model on a remote instance or evaluate an already trained model file in a consistent form

In our project we choose to use scripts instead of Jupyter Notebooks for the reasons cited above, however notebooks could still be used as a form of experimentation of models or processes and the script as a more 'definitive' form

CASE STUDY 2

https://github.com/ghimiresunil/Machine-Learning-Project-Structure

Steps involved in making a project Structure

Step 01: Make sure that you have latest python and pip installed in your system.

Step 02: Create a sample repository on github.com (For example: Machine-Learning-Project)

Step 03: Clone the repo in your local system git clone <url>

Step 04: Change directory to new directory 'Machine-Learning-Project' cd Machine-Learning-Project

Step 05: Create and activate virtaul environment

```
Example 01:

- Create Virtual Environment
    python -m venv venv_machine_learning_project

- Activate Created Virtual Environment
    For unix based system -> source ./venv_machine_learning_project/bin/activate
    For windows -> ./venv_machine_learning_project/Scripts/activate

Example 02:

- Create Virtual Environment
    conda create -n venv_machine_learning_project

- Activate Created Virtual Environment
    conda activate venv_machine_learning_project
```

Step 06: Follow the below directory structure for your project

```
├── Machine Learning Project Structure <- Project Main Directory
|   |── api <- Consists of scripts which serialize the API calls and act as a endpoint faciliating for
|   ├── data <- data in different format
|   |   ├── external <- data from third party source
|   |   ├── interim <- Intermediate data that has been transformed
|   |   ├── processed <- The final, canonical data sets for modeling
|   |   ├── raw <- The original, immutable data dump
|   ├── evaluation
|   |   ├── evaluate_model_01.py <- Different Matries used to evaluate the model
|   |   ├── evaluate_model_02.py <- Different Matries used to evaluate the model
|   ├── examples
|   |   ├── feature_01.md <- It consists of doc and example showing how we can use the project, differe
|   |   ├── feature_02.md <- It consists of doc and example showing how we can use the project, differe
|   ├── notebooks <- All the ipython notebooks used for EDA, visualization and verification of concept
|   ├── src
|   |   ├── dataset
|   |   |   ├── download_dataset.py <- Scripts to download the dataset or ccesing dataset from data fol
|   |   ├── model
|   |   |   ├── train_model.py <- Scripts to train the model
|   |   |   ├── test_model.py <- scripts to test the model
|   |   |   ├── predict_model.py <- Scripts to predict the model
|   |   ├── network
|   |   |   ├── approach_01.py <- Neural network schema
|   |   ├── weights
|   |   |   ├── utils.py.py <- folder to save weights
|   |   ├── visualization
|   |   |   ├── visaulization_model.py <- Scripts to visualize the model
|   |   ├── utils.py <- different utils functions
|   |   ├── project.py <- project pipeline
```

```
|   ├── project_cli <- Scripts which faciliates Command line interface for taining, testing and other f
|   |   ├── train_cli.py
|   |   ├── test_cli.py
|   ├── task <- Contains batch script which can be used for downloading files from web or batch to auto
|   |   ├── download.sh
|   |   ├── lint.sh
|   |   ├── est_api.sh
|   ├── training <- Contains all experiments preperation, way on auto running experiments and updating
|   |   ├── experiment
|   |   |   ├── utils.py
|   |   ├── prepare_experiment.py
|   |   ├── run_experiment.py
|   |   ├── update_metadata.py
|   ├── sqs
|   |   ├── SQSSender.py <- sending message to Amazon SQS
|   ├── aws
|   |   ├── download_files.py  <- uploading and downloading files from Amazon S3 Bucket
|   ├── config.ini <- Contains configuration information of project
|   ├── .pre-commit-config.yaml <- identifying simple issues before submission to code review
|   ├── .gitignore <- tells Git which files to ignore when committing your project to the GitHub reposi
|   ├── .env <- used to hide the confidential data like AWS Screte Key, AWS  Access Key, S3 Bucket Name
|   ├── Dockerfile <- This helps in dockerizing whole system
|   ├── requirements.txt <- requirements files contains all the module used while building the project.
|   ├── application.py <- python module that processes event i.e. function is invoked, Lambda runs the
|   ├── README.md <- The top-level README for developers using this project
```

Note: The data folder and .env file won’t appear in github. It will be in your local folder. This is not pushed to githhub as it will be in the ignore list (.gitignore file). If you want to checkin that also, just comment out in .gitignore file and add the data folder to github.